Assignment 2: Time series graphics

Data Forecasting. Plots made with Python (pandas + matplotlib); the full code is in the appendix and in `analysis.py`.

Before we start

Replacing a dataset to have seasonality. None of the three Assignment 1 series is seasonal. NVIDIA and S&P 500/VIX are financial market prices, and the euro-area GDP was seasonally adjusted by Eurostat (code SCA), which means the seasonality had been removed on purpose. As the assignment asks, dataset 3 is replaced by the same GDP series, not seasonally adjusted (code NSA): same source (Eurostat table `namq_10_gdp`, real GDP index 2020 = 100, euro area EA20), same 120 quarters (1996 Q3 to 2026 Q2), no missing values. It was downloaded on 21 October 2026 with `download_gdp.py` and saved as `03_euro_area_real_gdp_nsa_quarterly.csv`. This series is used from now on.

Number of lags. All lag plots use 12 lags (at least 9 were required): 12 lags cover one year of monthly data and three years of quarterly data, so a yearly seasonal pattern would show up. For the weekly NVIDIA data, 12 lags cover about 3 months (a yearly pattern would need lag 52), so its seasonality is judged from the line plot.

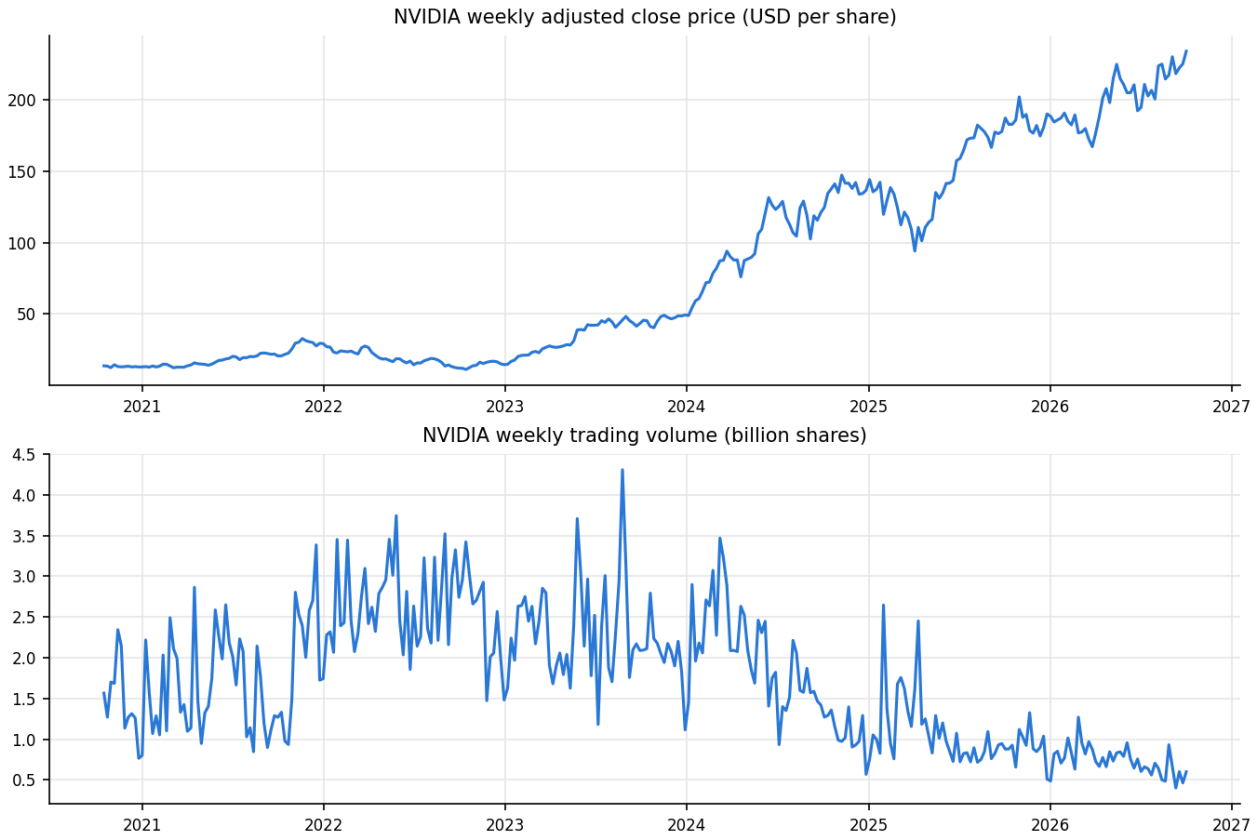
Words used:

- Trend = a long-term increase or decrease.
- Seasonality = a pattern that repeats at a fixed period (e.g. every year).
- Cycle = rises and falls that are not of fixed length (e.g. recessions).
- Stationary = the series looks the same no matter when you look at it: no trend, no seasonality, roughly constant spread.

1 NVIDIA stock (weekly, 312 weeks, Oct 2020 to Oct 2026)

Two variables: adjusted close price (USD) and weekly trading volume (shown in billions of shares).

Line plot



Price:

- Trend: strong upward trend, from about \$13 in 2020 to about \$234 in 2026, almost all of it after the start of 2023. Before that there is a rise in 2021 and a fall in 2022 (from about \$33 to \$11).
- Seasonality: none; nothing repeats at the same time each year.
- Cycle: no clear cycle. One full boom and bust (Nov 2021 to Oct 2022, −66%); the later falls (−22% in summer 2024, −36% from Nov 2024 to Apr 2025) were short dips inside the uptrend.
- Variance: the swings get larger as the price gets higher (e.g. the fall from about \$147 in November 2024 to \$94 in April 2025), so the variance is not constant.
- Stationary: no (trend and growing variance).

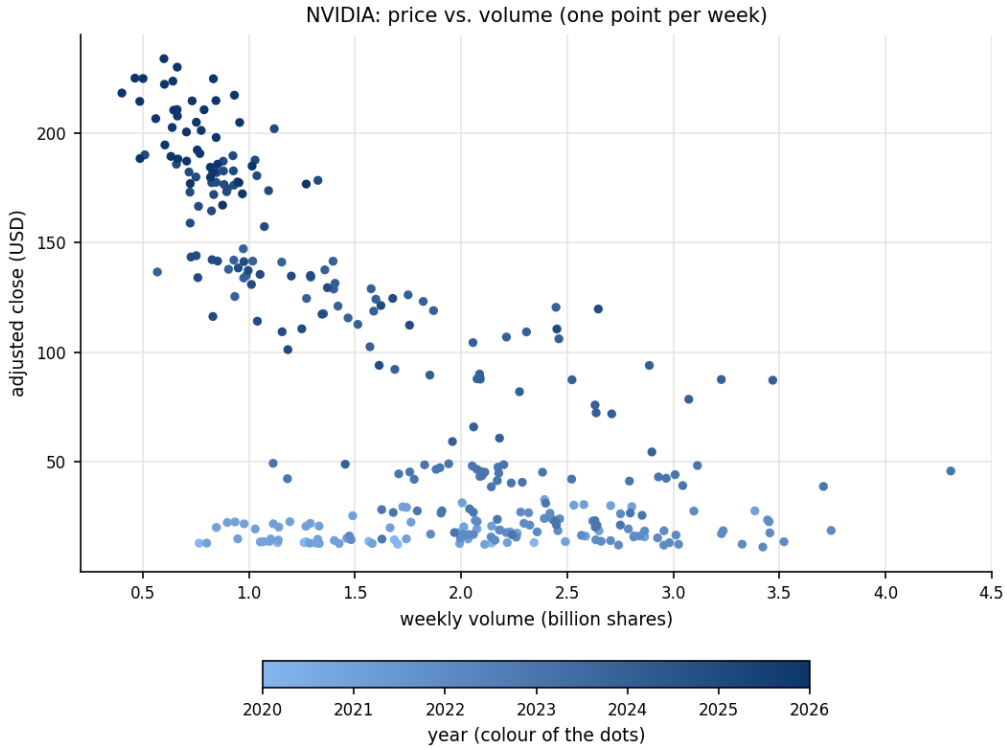
Volume:

- Trend: no steady trend, but the level changes: higher in 2022–2023 (around 2–3 billion shares a week), then a clear decline since mid-2024 down to about 0.5–1 billion.
- Seasonality: none visible.
- Cycle: no clear cycle.
- Variance: very noisy from week to week, with single-week spikes.
- Stationary: no, because the average level changes over time.

Seasonal plots

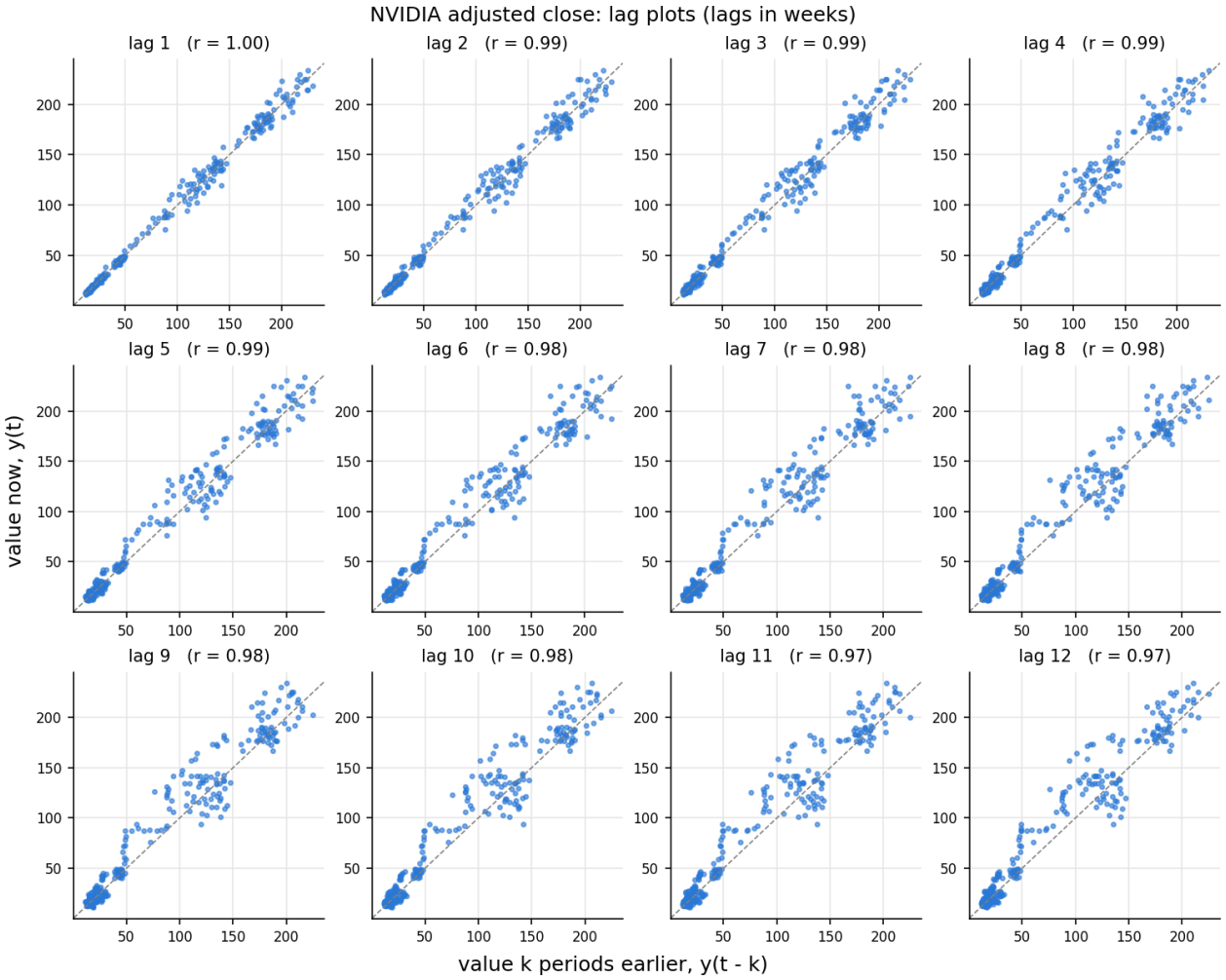
Not made: the line plot shows no seasonality in either variable.

Scatter plot (price vs. volume)



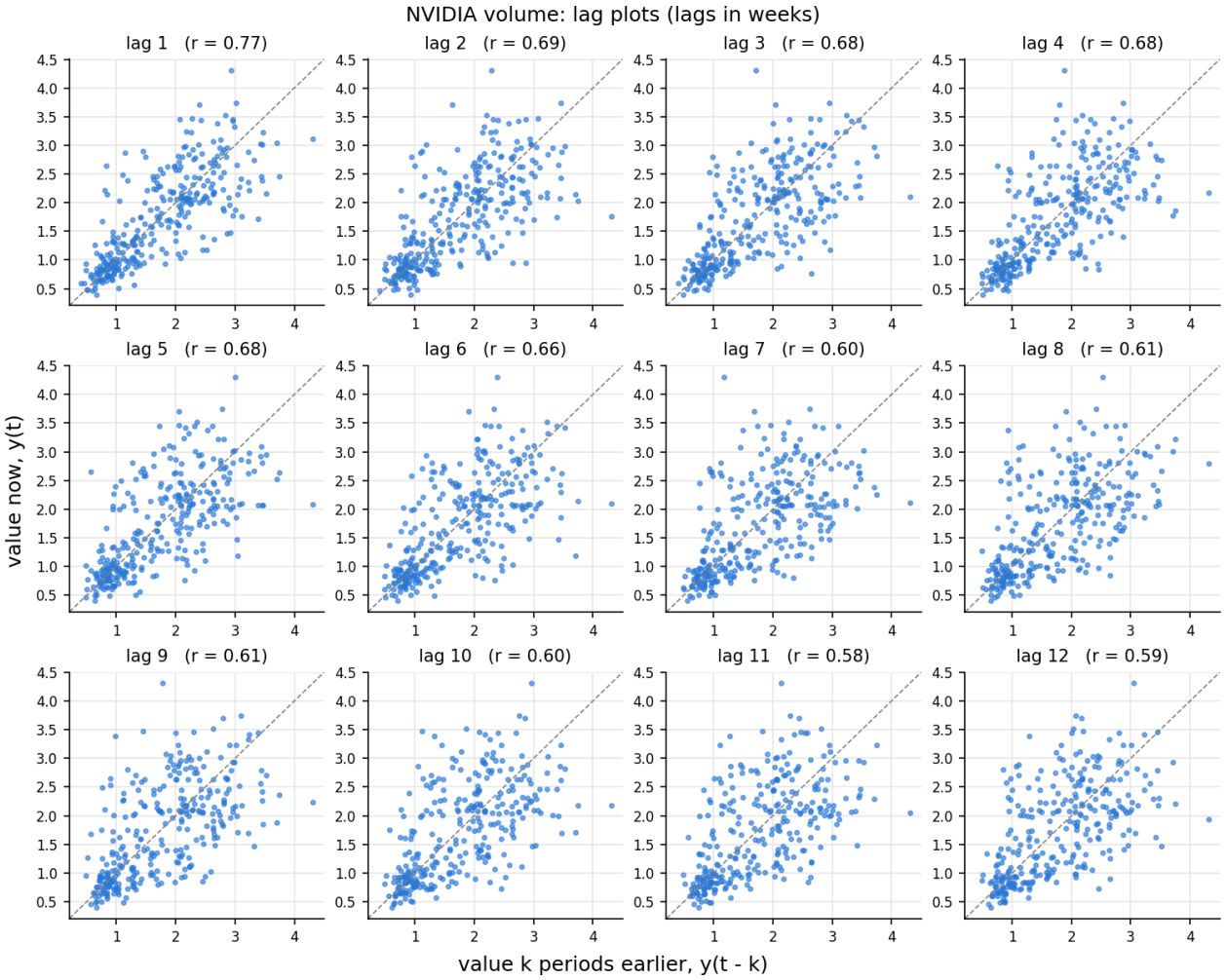
- With two variables there is only one pair. The colour shows the year (light = 2020, dark = 2026).
- There is a negative relationship (correlation -0.68): high-price weeks have low volume.
- The colours show that this is mostly about time: in 2020–2023 (light dots) the price was low and the volume varied a lot, while in 2025–2026 (dark dots) the price is high and the volume is low.
- Both variables trend over time in opposite directions, so the plot does not prove that one causes the other.
- One likely reason: volume is counted in shares, and at a higher price fewer shares are needed for the same amount of money.

Lag plots: price



- At every lag the dots lie very close to the diagonal (r goes only from 1.00 at lag 1 to 0.97 at lag 12): the price this week is almost the same as the price a few weeks ago.
- This strong, slowly fading dependence is typical of a trending, non-stationary series.
- At larger lags the cloud spreads out and moves above the diagonal, because the price now is usually higher than k weeks ago (the upward trend).
- No lag stands out, so there is no repeating pattern within 12 weeks.

Lag plots: volume

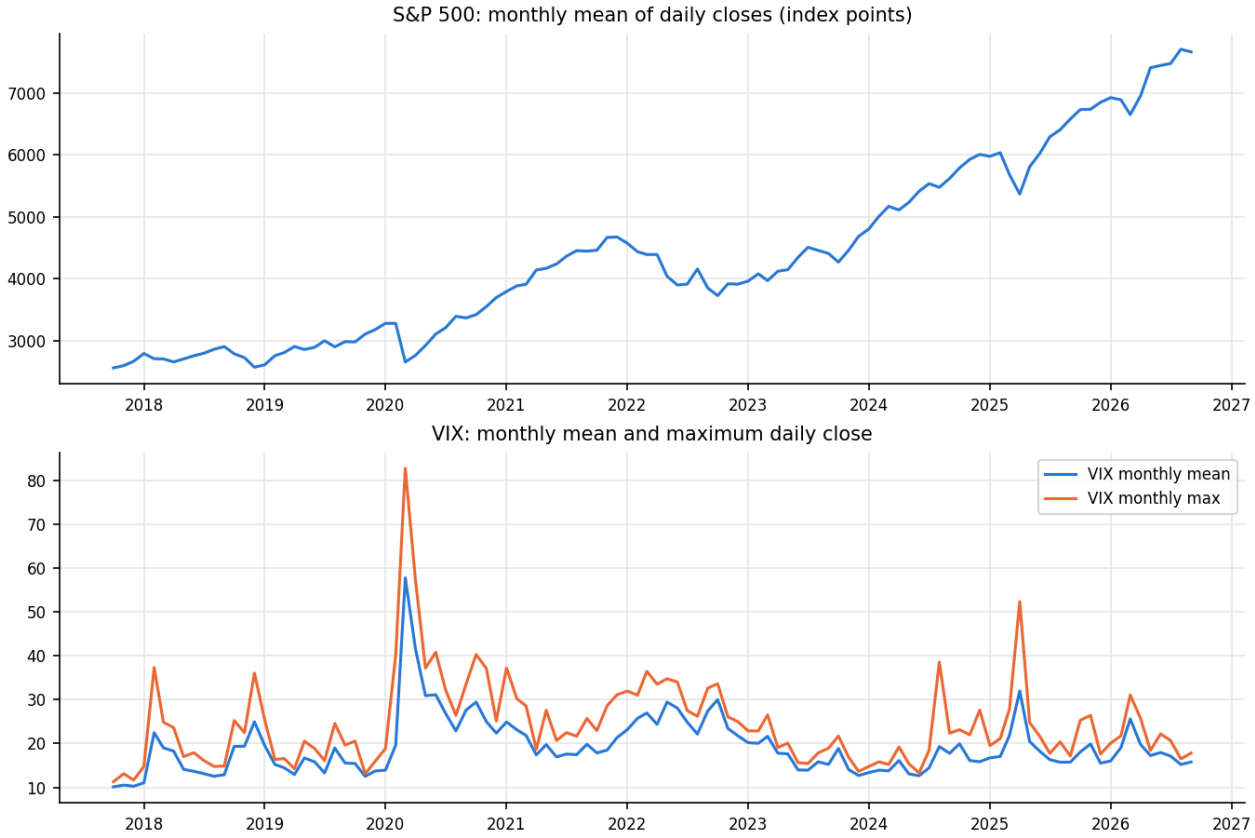


- A clear but much weaker positive relationship: r = 0.77 at lag 1, falling slowly to about 0.6 at lag 12.
- The wide cloud shows the large week-to-week noise.
- The slow decline in r comes from the long periods of high volume (2022–2023) and low volume (2025–2026): weeks inside the same period resemble each other.
- No lag stands out, so there is no repeating pattern within 12 weeks.

2 S&P 500 and VIX (monthly, 108 months, Oct 2017 to Sep 2026)

Three variables: S&P 500 monthly mean close, VIX monthly mean close, VIX monthly maximum daily close. VIX measures expected stock-market volatility ("fear index").

Line plot



S&P 500:

- Trend: clear upward trend, from about 2,600 to about 7,700 points.
- Seasonality: none; the falls listed below do not happen in the same month each year.
- Cycle: no regular cycle. The trend is interrupted by falls at irregular times: late 2018, the COVID crash in March 2020, a long decline during 2022, a drop in March–April 2025 and a smaller one in March 2026.
- Stationary: no (trend).

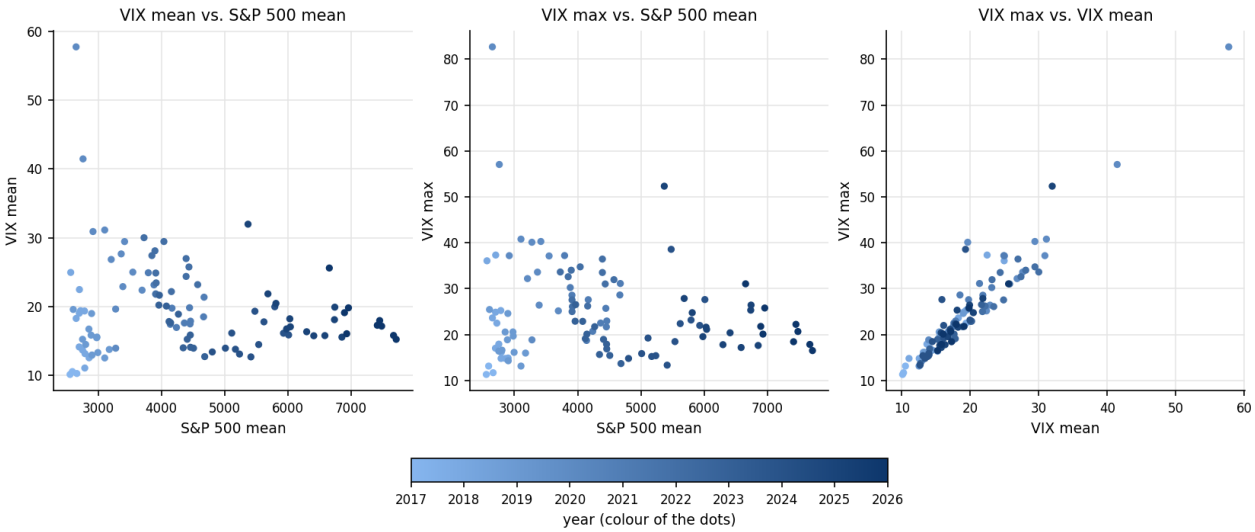
VIX (mean and max):

- Trend: none; the series keeps returning to about 15–20.
- Seasonality: none; the spikes happen in different months.
- Cycle: no regular cycle. Sudden spikes fade within a few months: early 2018, end of 2018, March 2020 (the largest: mean 57.7, max 82.7), August 2024 and April 2025. From mid-2020 to the end of 2022 VIX stays above its usual level (mostly 20–30) without big spikes.
- Mean vs. max: the maximum is always above the mean (by definition), and the gap is largest during spikes.
- Stationary: roughly yes, the closest to stationary of all the series: no trend, no seasonality, a stable usual level; only the spikes make the spread uneven.

Seasonal plots

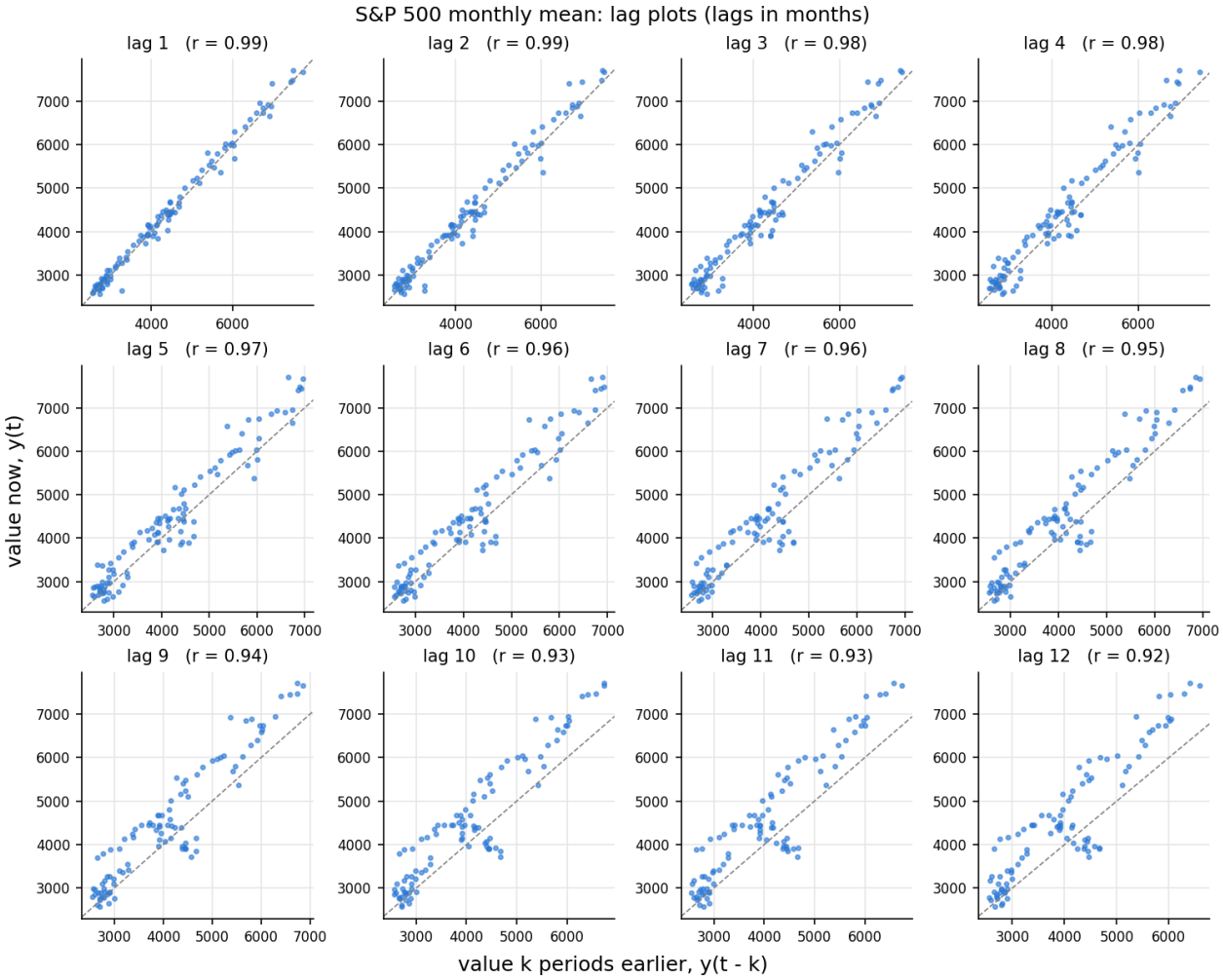
Not made: the line plot shows no seasonality in any of the three variables.

Pairwise scatter plots



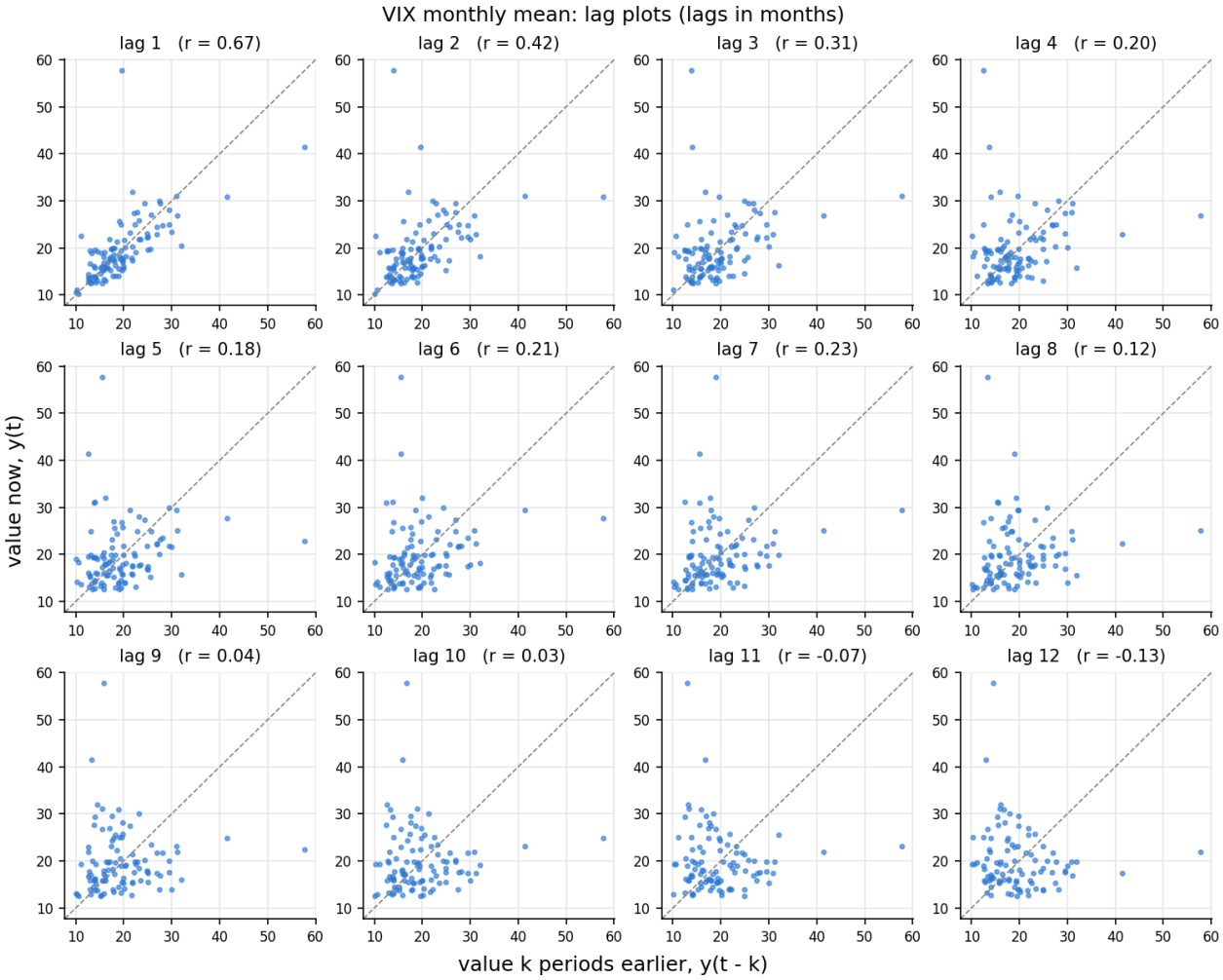
- All three pairs of variables; colour shows the year (light = 2017, dark = 2026).
- VIX mean vs. S&P 500 and VIX max vs. S&P 500 (left, middle): only a weak negative relationship (correlation −0.14 for both).
- The S&P 500 trends up over time while VIX does not, so the S&P level alone says little about VIX.
- But the highest VIX values occur in months when the S&P 500 had just fallen (March 2020 at about 2,650 points, April 2025 at about 5,370).
- VIX max vs. VIX mean (right): a strong positive, almost straight-line relationship (correlation 0.95), as expected, since both are computed from the same daily values.
- The single point at the top right is March 2020.

Lag plots: S&P 500



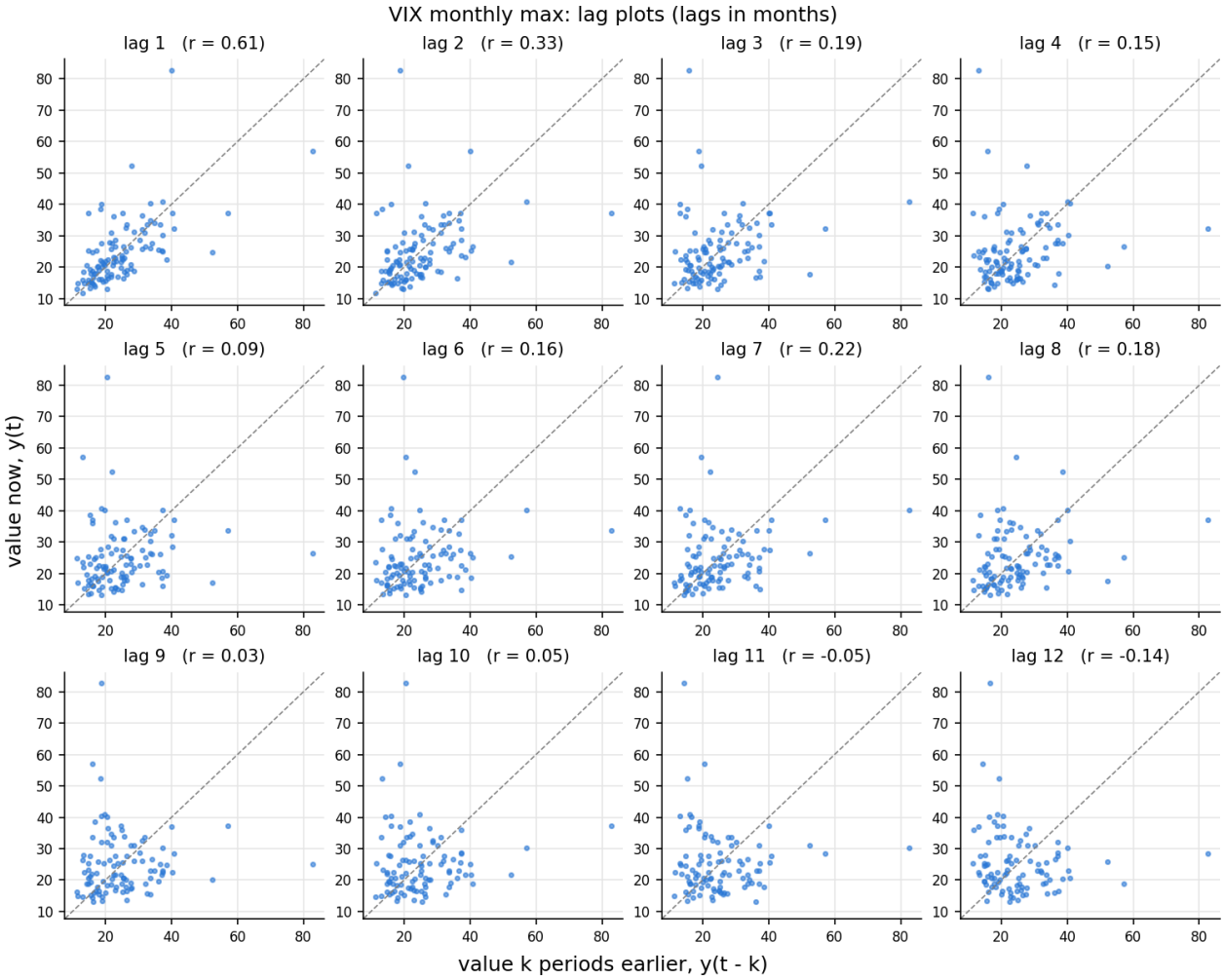
- Very similar to the NVIDIA price: dots close to the diagonal at all lags (r from 0.99 down to 0.92), a sign of a trending, non-stationary series.
- As the lag grows, most dots move above the diagonal (the index is higher than k months ago, i.e. the upward trend).
- The group of dots below the diagonal around 4,000–4,700 is 2022, when the index was lower than in the months before.
- No lag stands out (lag 12 is not special), so no seasonality.

Lag plots: VIX mean



- Very different from the S&P 500. At lag 1 there is a clear positive relationship (r = 0.67): a high-volatility month is usually followed by another fairly high one.
- But it fades fast: r = 0.42 at lag 2, about 0.2 at lags 4–7 and about 0 at lags 9–12.
- So VIX only "remembers" the last few months, which fits a stationary series that returns to its usual level after each spike.
- Lag 12 shows nothing special (no yearly pattern), so no seasonality.
- The isolated dots far from the cloud come from March–April 2020.

Lag plots: VIX max

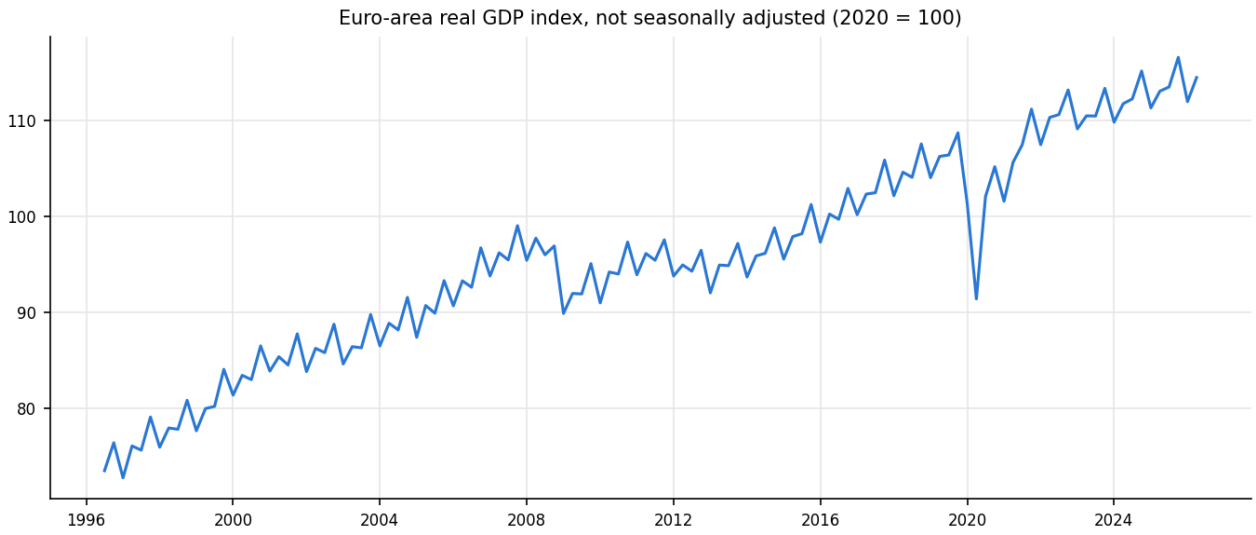


- Same picture as the VIX mean, slightly weaker: r = 0.61 at lag 1, about 0.1–0.2 at lags 3–8, near 0 at lags 9–12.
- The monthly maximum depends on a single day, so it is noisier than the monthly mean.
- Again a short memory, no seasonality.
- The dot at 82.7 is March 2020.

3 Euro-area real GDP, not seasonally adjusted (quarterly, 120 quarters, 1996 Q3 to 2026 Q2)

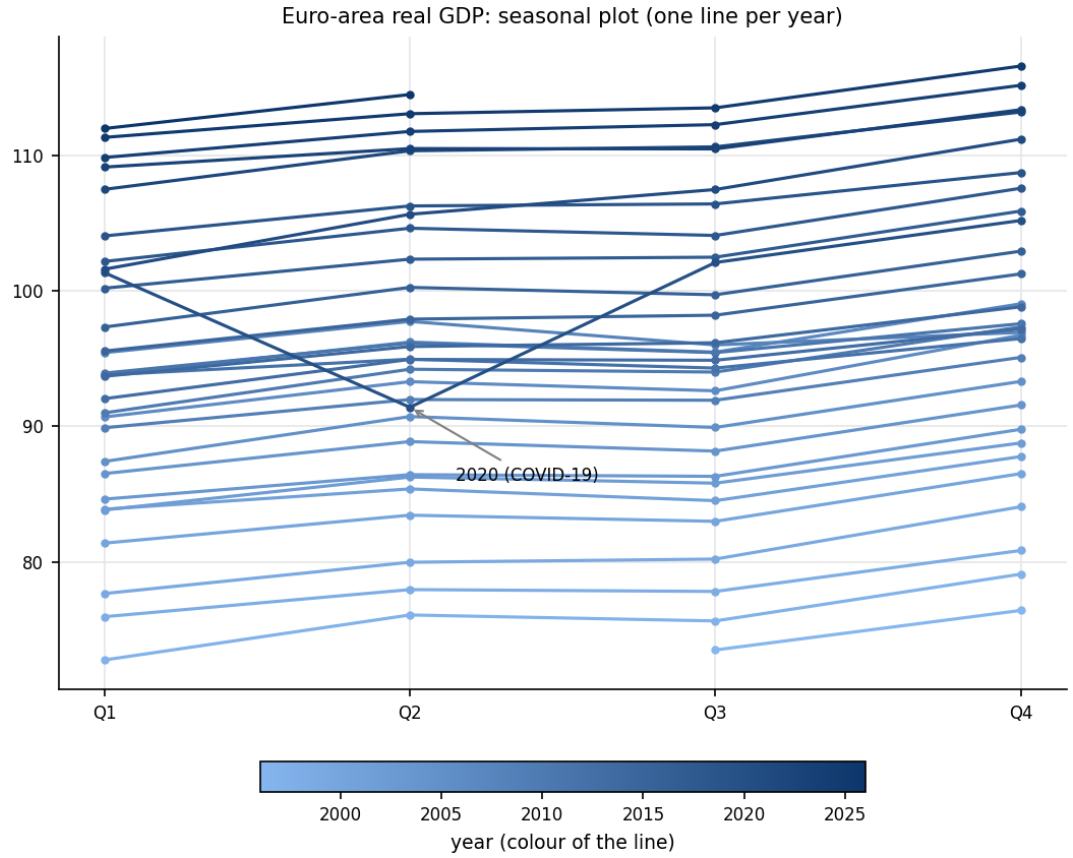
One variable: real GDP index (2020 = 100). This is the seasonal replacement described at the start.

Line plot



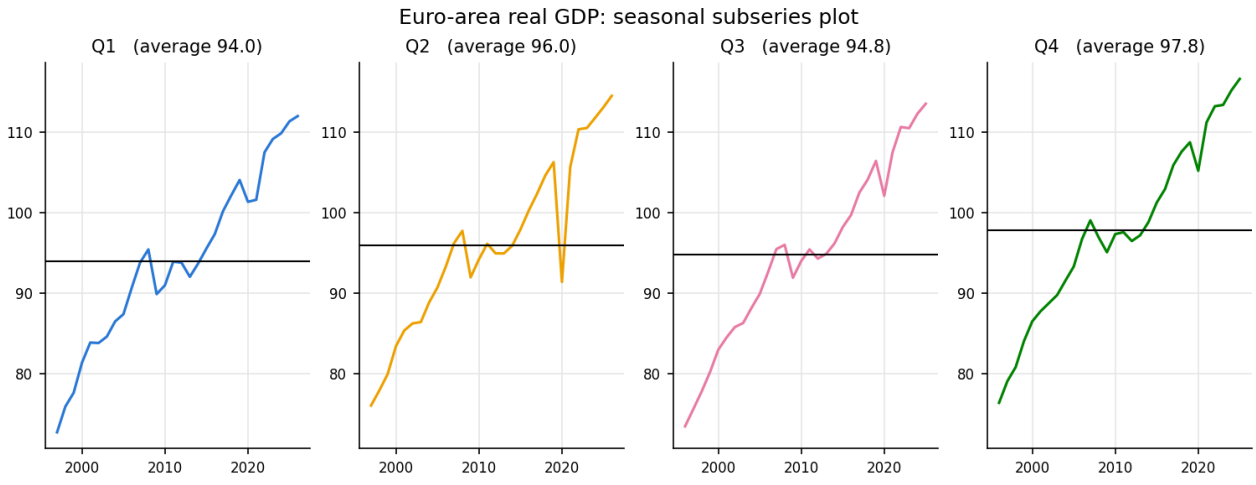
- Trend: GDP grows from about 74 to about 115 over 30 years; growth is faster before 2008 than after.
- Seasonality: a saw-tooth that repeats every year (every 4 quarters): low in Q1, high in Q4.
- Cycle: the business cycle shows as irregular downturns: the 2008–2009 financial crisis, flat or falling GDP around 2011–2013 (euro debt crisis) and the COVID-19 collapse in 2020 Q2 followed by a fast rebound. These downturns are not at fixed intervals, so they are a cycle, not seasonality.
- Stationary: no (trend and seasonality).

Seasonal plot



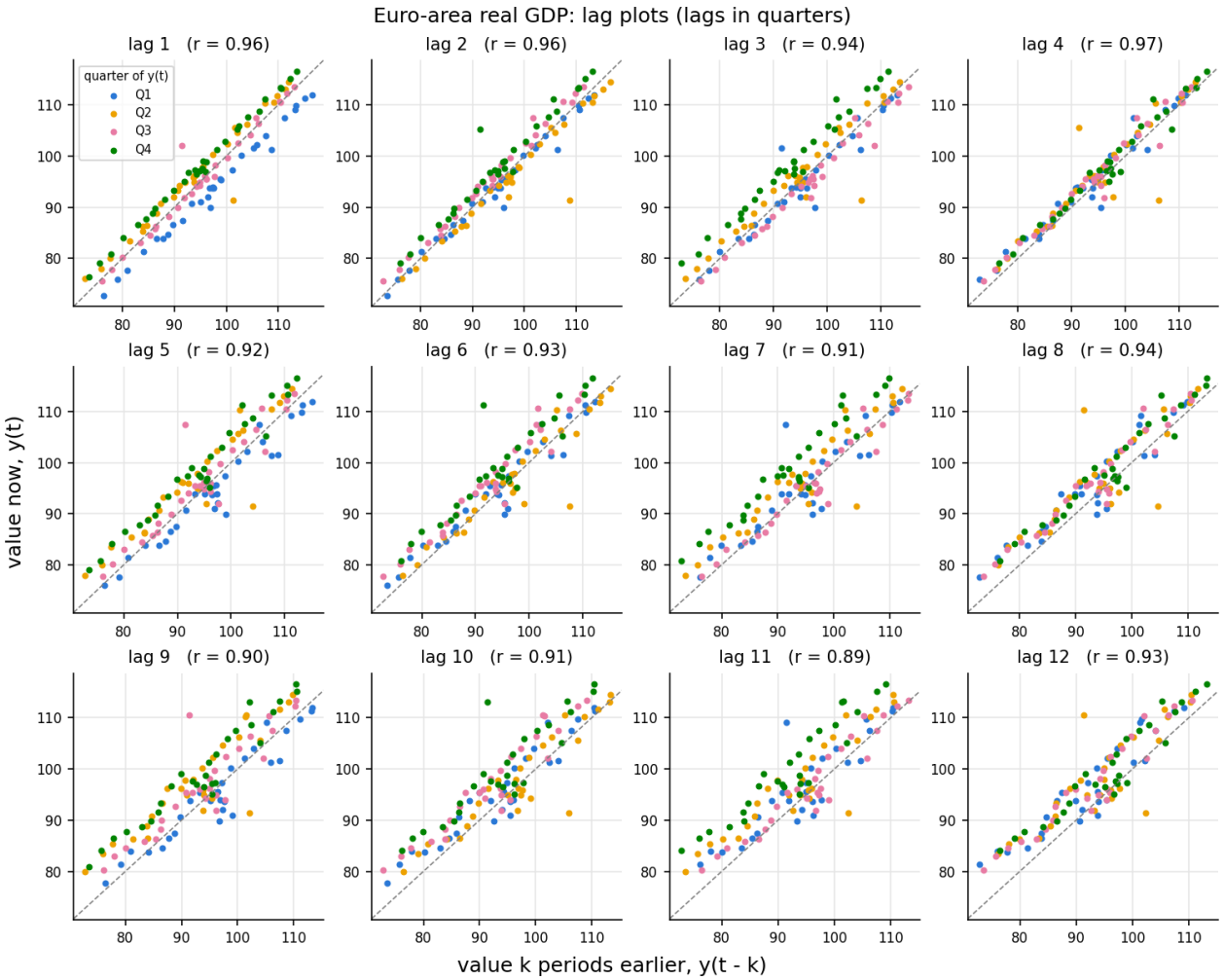
- Each line is one year (light = 1996, dark = 2026), plotted across its four quarters.
- Almost every line has the same shape: lowest in Q1, up in Q2, flat or slightly down in Q3, highest in Q4.
- In the 29 complete years (1997–2025), Q1 is the lowest quarter in 28 (all except 2020) and Q4 is the highest in 28 (all except 2008, the financial crisis).
- On average Q1 is about 2.4% below its year's average and Q4 about 3.0% above.
- The lines are stacked upwards because of the trend; the many lines bunched together around 90–98 are the slow-growth years 2008–2015.
- The one line that breaks the pattern is 2020, with its deep fall in Q2 (COVID-19 lockdowns).

Seasonal subseries plot



- Each panel shows one quarter across all years; the black line is that quarter's average.
- Q4 has the highest average (97.8) and Q1 the lowest (94.0), confirming the seasonal pattern. (The averages also contain the trend, so they are only a rough measure of the seasonal effect.)
- All four panels have the same shape (same trend, same dips in 2009 and 2020), which means the seasonal pattern is stable over time: it does not grow, shrink or change shape.
- The COVID-19 fall is by far the largest in Q2 (2020 Q2).

Lag plots



- Dots are coloured by the quarter of y(t). All lags have a high r (0.89 to 0.97) because of the trend.
- At lag 1 the colours separate: Q1 dots (blue) lie below the diagonal (Q1 is lower than the Q4 just before it), and Q4 dots (green) lie above (Q4 is higher than Q3).
- At lags 4, 8 and 12 (the same quarter one, two and three years earlier) all colours mix in a narrow band along the diagonal, and r is higher than at the neighbouring lags (lag 4: 0.97 vs. 0.94 at lag 3 and 0.92 at lag 5; lag 8: 0.94 vs. 0.91 at lag 7 and 0.90 at lag 9).
- Together, this is the signature of seasonality with period 4 quarters (one year).
- In every panel, two dots lie far from the diagonal, and both come from the single COVID-19 quarter 2020 Q2 (GDP 91.4). The dot below the line is 2020 Q2 itself as the value now (always orange). The dot above the line is the quarter k periods later, compared with that very low value; its colour changes with the lag (e.g. lag 1: 2020 Q3, pink; lag 4: 2021 Q2, orange).

4 Summary

Series	Trend	Seasonality	Cycle	Looks stationary?
NVIDIA price	yes, up	no	no (one boom/bust, short dips)	no
NVIDIA volume	level changes	no	no	no
S&P 500	yes, up	no	irregular falls	no
VIX mean / VIX max	no	no	irregular spikes	roughly yes
Euro-area GDP (NSA)	yes, up	yes, yearly	yes (recessions)	no

- Most of the series are not stationary because of a trend; to model them, differencing (or returns) would be needed. VIX is the only one that looks roughly stationary.
- The euro-area GDP (not seasonally adjusted) is the seasonal series that will be used from now on: a stable yearly pattern with Q1 low and Q4 high.
- In the multivariate datasets, the strongest relationship is VIX max vs. VIX mean. The other relationships (NVIDIA price vs. volume, S&P 500 vs. VIX) are weaker and mostly driven by time.

Appendix: code

How to run. From the assignment folder: `pip install -r requirements.txt`, then `python analysis.py`. It reads the three CSV files and saves all the plots in `figures/`. `download_gdp.py` was run once to create the new seasonal GDP CSV.

analysis.py

```
"""Assignment 2: plots for the three time series.

Run from this folder: python analysis.py
It reads the three CSV files and saves every plot as a PNG in the figures/ folder.

Plots made (as the assignment asks):
  - a line plot for every dataset
  - seasonal plot + seasonal subseries plot for the seasonal dataset (GDP)
  - pairwise scatter plots for the multivariate datasets (NVIDIA, S&P 500/VIX)
  - lag plots (lags 1 to 12) for every variable
"""

from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.colors import LinearSegmentedColormap

Path("figures").mkdir(exist_ok=True)

# ------------------------------------------------------------------ style
plt.rcParams.update({
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.axisbelow": True, # grid lines behind the data
    "grid.color": "#e6e6e6",
    "axes.titlesize": 10,
    "axes.labelsize": 9,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
})
BLUE = "#2a78d6"
ORANGE = "#eb6834"
# Light blue = old, dark blue = recent. Used whenever colour shows time.
YEAR_COLORS = LinearSegmentedColormap.from_list("years", ["#86b6ef", "#0d366b"])
# One fixed colour per quarter (checked to be colour-blind safe).
QUARTER_COLORS = {1: "#2a78d6", 2: "#eda100", 3: "#e87ba4", 4: "#008300"}


def save(fig, name):
    fig.savefig(f"figures/{name}.png", dpi=150)
    plt.close(fig)
    print(f"saved figures/{name}.png")


def lag_plot(series, title, name, color_by_quarter=False):
    """12 small scatter plots, one per lag k = 1..12.

    Each point is one time t: x = the value k periods earlier, y = the value now.
    Points close to the dashed diagonal mean "the value now is about the same as k periods ago".
    r is the correlation between the two (close to 1 = strong relationship).
    """
    fig, axes = plt.subplots(3, 4, figsize=(10, 8), layout="constrained")
    for k, ax in zip(range(1, 13), axes.flat):
```

```
        lagged = series.shift(k) # the value k periods earlier, lined up with the current one
        if color_by_quarter:
            for q, color in QUARTER_COLORS.items():
                now_in_q = series.index.quarter == q
                ax.scatter(lagged[now_in_q], series[now_in_q], s=8, color=color, label=f"Q{q}")
        else:
            ax.scatter(lagged, series, s=6, color=BLUE, alpha=0.6)
        # Diagonal line y = x (drawn through the smallest value so the axes don't stretch to 0)
        ax.axline((series.min(), series.min()), slope=1, color="grey", linestyle="--", linewidth
            =0.8)
        ax.set_title(f"lag {k} (r = {series.autocorr(k):.2f})")
    if color_by_quarter:
        axes[0, 0].legend(title="quarter of y(t)", fontsize=7, title_fontsize=7)
    fig.supxlabel("value k periods earlier, y(t - k)")
    fig.supylabel("value now, y(t)")
    fig.suptitle(title)
    save(fig, name)


# ------------------------------------------------------------- load the data
nvda = pd.read_csv("01_nvda_weekly.csv", parse_dates=["date"], index_col="date")
market = pd.read_csv("02_sp500_vix_monthly.csv", parse_dates=["date"], index_col="date")
gdp = pd.read_csv("03_euro_area_real_gdp_nsa_quarterly.csv", parse_dates=["date"], index_col="date")


price = nvda["nvda_adjusted_close_usd"]
volume = nvda["nvda_volume_shares"] / 1e9 # in billions of shares, easier to read
sp500 = market["sp500_mean_close"]
vix_mean = market["vix_mean_close"]
vix_max = market["vix_max_close"]
gdp = gdp["real_gdp_index"]


# ================================================================ 1. NVIDIA (weekly)
# 1a. Line plot of both variables
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), layout="constrained")
ax1.plot(price, color=BLUE)
ax1.set_title("NVIDIA weekly adjusted close price (USD per share)")
ax2.plot(volume, color=BLUE)
ax2.set_title("NVIDIA weekly trading volume (billion shares)")
save(fig, "1a_nvda_line")

# 1b. Scatter plot: price against volume (only one pair, since there are two variables)
fig, ax = plt.subplots(figsize=(7, 5.2), layout="constrained")
points = ax.scatter(volume, price, c=nvda.index.year, cmap=YEAR_COLORS, s=12)
# Colour key under the plot (horizontal), so it is not read against the price axis
fig.colorbar(points, ax=ax, label="year (colour of the dots)", orientation="horizontal", shrink=0.6)

ax.set_xlabel("weekly volume (billion shares)")
ax.set_ylabel("adjusted close (USD)")
ax.set_title("NVIDIA: price vs. volume (one point per week)")
save(fig, "1b_nvda_scatter")

# 1c, 1d. Lag plots
lag_plot(price, "NVIDIA adjusted close: lag plots (lags in weeks)", "1c_nvda_lag_price")
lag_plot(volume, "NVIDIA volume: lag plots (lags in weeks)", "1d_nvda_lag_volume")


# ================================================================ 2. S&P 500 and VIX (monthly)
# 2a. Line plot. S&P 500 and VIX have very different scales, so they get separate panels.
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), layout="constrained")
ax1.plot(sp500, color=BLUE)
ax1.set_title("S&P 500: monthly mean of daily closes (index points)")
ax2.plot(vix_mean, color=BLUE, label="VIX monthly mean")
ax2.plot(vix_max, color=ORANGE, label="VIX monthly max")
```

```
ax2.set_title("VIX: monthly mean and maximum daily close")
ax2.legend(fontsize=8)
save(fig, "2a_market_line")

# 2b. Pairwise scatter plots: every pair of the three variables
pairs = [
    (sp500, vix_mean, "S&P 500 mean", "VIX mean"),
    (sp500, vix_max, "S&P 500 mean", "VIX max"),
    (vix_mean, vix_max, "VIX mean", "VIX max"),
]
fig, axes = plt.subplots(1, 3, figsize=(11, 4.6), layout="constrained")
for (x, y, x_label, y_label), ax in zip(pairs, axes):
    points = ax.scatter(x, y, c=market.index.year, cmap=YEAR_COLORS, s=12)
    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    ax.set_title(f"{y_label} vs. {x_label}")
fig.colorbar(points, ax=axes, label="year (colour of the dots)", orientation="horizontal", shrink
    =0.4)
save(fig, "2b_market_scatter")

# 2c-2e. Lag plots
lag_plot(sp500, "S&P 500 monthly mean: lag plots (lags in months)", "2c_sp500_lag")
lag_plot(vix_mean, "VIX monthly mean: lag plots (lags in months)", "2d_vix_mean_lag")
lag_plot(vix_max, "VIX monthly max: lag plots (lags in months)", "2e_vix_max_lag")


# ============================================================ 3. Euro-area real GDP, NOT
    seasonally adjusted (quarterly)
# 3a. Line plot
fig, ax = plt.subplots(figsize=(9, 3.8), layout="constrained")
ax.plot(gdp, color=BLUE)
ax.set_title("Euro-area real GDP index, not seasonally adjusted (2020 = 100)")
save(fig, "3a_gdp_line")

# 3b. Seasonal plot: one line per year, x axis = quarter of the year
first_year, last_year = gdp.index.year.min(), gdp.index.year.max()
fig, ax = plt.subplots(figsize=(7, 5.6), layout="constrained")
for year, values in gdp.groupby(gdp.index.year):
    shade = (year - first_year) / (last_year - first_year) # 0 = oldest year, 1 = newest
    ax.plot(values.index.quarter, values, color=YEAR_COLORS(shade), marker="o", markersize=3)
ax.set_xticks([1, 2, 3, 4], ["Q1", "Q2", "Q3", "Q4"])
ax.annotate("2020 (COVID-19)", xy=(2, gdp["2020-04-01"]), xytext=(2.15, 86), fontsize=8,
            arrowprops={"arrowstyle": "->", "color": "grey"})
year_scale = plt.cm.ScalarMappable(cmap=YEAR_COLORS, norm=plt.Normalize(first_year, last_year))
fig.colorbar(year_scale, ax=ax, label="year (colour of the line)", orientation="horizontal", shrink
    =0.6)
ax.set_title("Euro-area real GDP: seasonal plot (one line per year)")
save(fig, "3b_gdp_seasonal")

# 3c. Seasonal subseries plot: one panel per quarter, showing that quarter across all years.
# The black line is the average of that quarter.
fig, axes = plt.subplots(1, 4, figsize=(10, 3.8), layout="constrained")
for q, ax in zip([1, 2, 3, 4], axes):
    values = gdp[gdp.index.quarter == q]
    ax.plot(values.index.year, values, color=QUARTER_COLORS[q])
    ax.axhline(values.mean(), color="black", linewidth=1)
    ax.set_title(f"Q{q} (average {values.mean():.1f})")
fig.suptitle("Euro-area real GDP: seasonal subseries plot")
save(fig, "3c_gdp_subseries")

# 3d. Lag plots, coloured by quarter so the seasonal pattern is visible
lag_plot(gdp, "Euro-area real GDP: lag plots (lags in quarters)", "3d_gdp_lag", color_by_quarter=
    True)
```

download_gdp.py

```
"""Download the seasonal replacement for dataset 3 and save it as a CSV.

Assignment 1 used euro-area real GDP *seasonally adjusted* (Eurostat code SCA),
which means Eurostat had already removed the seasonality. Here we download the
same series *not seasonally adjusted* (code NSA), so the seasonality is still in it.
Same quarters as before: 1996 Q3 to 2026 Q2 (120 quarters).

Run once: python download_gdp.py
"""

import pandas as pd

# Eurostat table namq_10_gdp, filtered to:
# Q = quarterly, CLV_I20 = real GDP index (2020 = 100), NSA = not seasonally adjusted,
# B1GQ = gross domestic product, EA20 = euro area (20 countries).
URL = (
    "https://ec.europa.eu/eurostat/api/dissemination/sdmx/2.1/data/namq_10_gdp/"
    "Q.CLV_I20.NSA.B1GQ.EA20?format=SDMX-CSV&startPeriod=1996-Q3&endPeriod=2026-Q2"
)

raw = pd.read_csv(URL)

# Keep only the two columns we need: the quarter ("1996-Q3") and the value.
gdp = pd.DataFrame({
    # "1996-Q3" -> 1996-07-01, the first day of the quarter (same style as the other CSVs).
    "date": pd.PeriodIndex(raw["TIME_PERIOD"].str.replace("-", ""), freq="Q").start_time,
    "real_gdp_index": raw["OBS_VALUE"],
})

assert len(gdp) == 120 and gdp["real_gdp_index"].notna().all()
gdp.to_csv("03_euro_area_real_gdp_nsa_quarterly.csv", index=False, date_format="%Y-%m-%d")
print(gdp.head(), gdp.tail(), sep="\n...\n")
```